Ejercicio 15

a) El algoritmo se llama exponenciación binaria. ¿Cómo funciona? En vez de multiplicar por por sí mismo m veces, se escribe m en binario. Se va elevando x al cuadrado y se multiplica en el resultado solo cuando el bit correspondiente vale 1. Pasos:

1. Inicializar resultado = 1.
2. Mientras m > 0:
    - Si m es impar, resultado = (resultado · x) mod n.
    - x = (x · x) mod n.
    - m = m ÷ 2 (parte entera).
3. El resultado final es x^m mod n.

Una de las ventajas es que se hacen del orden de $\log_2(m)$ multiplicaciones en lugar de m. Los números nunca crecen más que n, porque se aplica el módulo en cada paso. Sin esto, RSA con claves grandes sería inviable. Por ejemplo: 3^{13} mod 7. Como $13 = 1101_2$, la secuencia de pasos da 3^{13} mod 7 = 3. Python ya trae esto incorporado como `pow(x, m, n)`.

b) Algoritmos para factorizar un número N:

Los tres más usados:

- División por tentativa: prueba dividir por todos los números hasta √N. Es simple pero lento para números grandes.
- Fermat: el que usamos arriba. Es muy rápido si p y q están cerca.
- Pollard rho: más eficiente para números grandes y también simple de programar.

Algoritmo	Idea	Cuándo sirve
División por tentativa	Dividir N por todos los números hasta √N	Números chicos; muy lento para los grandes
Fermat	Buscar $N = a^2 - b^2 = (a-b)(a+b)$	Cuando los factores p y q están cercanos
Pollard rho	Usa una secuencia pseudoaleatoria y busca ciclos con el MCD	Números medianos; más eficiente que los anteriores
Criba cuadrática / criba general del cuerpo de números	Buscan relaciones entre cuadrados módulo N	Números muy grandes; los usados en la práctica

La seguridad de RSA se basa en que factorizar un N muy grande (p·q con primos grandes) es computacionalmente inviable. Pero si p y q son pequeños o están muy cerca, algoritmos simples como Fermat lo rompen en poco tiempo.